

Lab 03 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde stramien: het antwoord, het mechanisme, de nuance of valkuil, en een voorbeeld dat je zelf aan de terminal kunt controleren.

Vraag 1 — Drie gedragingen, één regel

Antwoord. De regel: **een container leeft precies zolang zijn PID 1**. Het standaardcommando van `alpine` is `/bin/sh`; zonder standaardinvoer leest de shell meteen een bestandseinde en sluit hij af → de container stopt. `nginx` draait op de voorgrond en eindigt nooit uit zichzelf → de container blijft leven, en blokkeert je terminal omdat je geen `-d` meegaf. `-it alpine sh` geeft de shell een open invoer en een terminal → hij wacht op je commando's, dus de container leeft tot jij `exit` typt.

Waarom. De engine doet niet meer dan een proces starten in namespaces en wachten tot het eindigt. Het begrip "dienst" bestaat voor hem niet: wat de container in leven houdt, is een proces dat niet eindigt.

Nuance. `podman run nginx` zonder `-d` betekent niet dat nginx "anders" draait. Het draait identiek; alleen hangt jouw terminal aan zijn uitvoer. `Ctrl+C` stuurt dan `SIGINT` naar PID 1 en stopt het.

Voorbeeld.

```
podman run alpine;           podman ps -a -l --format '{{.Status}}'    # Exited (0)
podman run -d nginx:alpine;  podman ps -l --format '{{.Status}}'    # Up
podman run -it alpine sh -c 'echo "ik leef zolang je wilt"; exit 7'; echo $?    # 7
```

Vraag 2 — De `&` die doodt

Antwoord. Het script is PID 1. `java ... &` start Java op de achtergrond en gaat meteen verder; `echo` wordt uitgevoerd; het script bereikt zijn laatste regel en eindigt met code `0`. Zodra PID 1 dood is, doodt de kernel de hele rest van de namespace, Java inbegrepen. De correctie: start Java op de voorgrond, als laatste regel, **en** met `exec`:

```
#!/bin/sh
echo "API gestart"
exec java -jar /app/api.jar
```

Waarom. `exec` vervangt de shell door Java, dat zo PID 1 wordt: het leeft zolang het wil en ontvangt `SIGTERM` rechtstreeks. Zonder `exec` maar ook zonder `&` zou het script op Java wachten (de container zou blijven leven), maar zelf PID 1 blijven — en `SIGTERM` niet doorgeven (vraag 3 van lab 04).

Nuance. Code `0` zet je op het verkeerde been: vanuit het script bekeken is alles "goed gegaan". Dit is een container die faalt zonder foutmelding — een *restart policy* `on-failure` zou hem niet eens herstarten.

Voorbeeld.

```
podman run --rm -v "$PWD":/s alpine /s/demarrage-casse.sh      # komt meteen terug
podman run -d --name ok -v "$PWD":/s alpine /s/demarrage-correct.sh && podman top ok    # sleep
als PID 1
```

Vraag 3 — Tien seconden en 137

Antwoord. `sleep` is PID 1, en de Linux-kernel laat PID 1 elk signaal negeren waarvoor het zelf geen handler installeerde. `sleep` installeert er geen, dus `SIGTERM` wordt genegeerd. Podman wacht de respijtp periode af (10 s), meldt dat het overschakelt op `SIGKILL` — dat kan niemand negeren — en het proces wordt gedood: code `128 + 9 = 137`. `143` (`128 + 15`) zie je alleen wanneer `SIGTERM` het proces daadwerkelijk beëindigd heeft.

Waarom. Die bescherming van PID 1 bestaat zodat een onhandige `kill -TERM 1` niet een hele machine neerhaalt. In een container werkt ze tegen je.

Nuance. Dit is niet typisch voor `sleep`: elk programma zonder `SIGTERM`-handler gedraagt zich zo als PID 1 — ook een shellsript, of een `java` met een shell ervoor. De waarschuwing die Podman toont (`resorting to SIGKILL`) is goud waard: Docker doodt zwijgend.

Voorbeeld.

```
podman run --rm alpine sh -c 'kill -TERM 1; echo overleefd'    # "overleefd": PID 1 negeerde
zijn eigen TERM
podman run -d --name v alpine sleep 300; time podman stop v    # 10 s, code 137
```

Vraag 4 — Wat `--init` verandert

Antwoord. `--init` zet `podman-init` (een binary van enkele KB, `catatonit`) neer als PID 1; `sleep` wordt zijn kind, PID 2. `podman-init` doet precies twee dingen: signalen doorgeven aan zijn kind en zombies opruimen. Bij de `stop` ontvangt het `SIGTERM` en geeft het door aan `sleep`, dat geen PID 1 meer is — dus geldt de standaardactie: het sterft. Code `143`, onmiddellijk. `podman exec` wacht `ps` toont `1 podman-init` en daaronder `2 sleep`.

Waarom. De kernelbescherming geldt alleen voor PID 1. Door je programma naar PID 2 te verschuiven, gedraagt het zich weer normaal tegenover signalen.

Nuance. `--init` is een pleister: het maakt je applicatie niet in staat om netjes af te sluiten, alleen om *netjes gedood te worden*. Een Spring Boot-API behandelt `SIGTERM` zelf; ze heeft geen `--init` nodig, ze moet het signaal **ontvangen** (exec-vorm). `--init` blijft wél nuttig voor images die meerdere processen starten en zombies produceren.

Voorbeeld.

```
podman run --rm --init alpine ps -o pid,comm    # 1 podman-init, 2 ps
```

Vraag 5 — `-i` zonder `-t`, `-t` zonder `-i`

Antwoord. `-i` houdt `stdin` open en verbonden met je toetsenbord; `-t` wijst een pseudo-terminal toe (prompt, echo, toetsafhandeling). Bij `podman run -t alpine sh` zie je een prompt, maar `stdin` is niet verbonden: je toetsaanslagen komen nergens aan, `ls` doet niets, en de container blijft hangen tot je hem vanuit een andere terminal doodt (`podman rm -f -t 0`). Bij

`podman run -i alpine sh` krijg je geen prompt en geen echo, maar wat je typt komt wél aan: `ls` wordt uitgevoerd en toont zijn resultaat — kaal, maar het werkt.

Waarom. Het zijn twee onafhankelijke kanalen: `-i` gaat over de gegevensstroom, `-t` over de presentatie. Een shell heeft alleen `-i` nodig om te werken; `-t` maakt hem aangenaam in gebruik.

Nuance. `-i` alleen is de vorm voor scripts: `echo "SELECT 1" | podman exec -i db psql -U app` werkt, terwijl het met `-t` zou falen (`the input device is not a TTY`). Een klassieke CI-bug.

Voorbeeld.

```
echo 'echo "ontvangen: $((6*7))"' | podman run -i --rm alpine sh      # ontvangen: 42 – zonder
prompt
podman run -it --rm alpine sh                                         # prompt "/ #", Ctrl+D om
eruit te gaan
```

Vraag 6 — `attach` en `Ctrl+C`

Antwoord. `attach` koppelde zijn terminal aan de stromen van **PID 1** — de API zelf. `Ctrl+C` stuurde `SIGINT` naar dat proces; het stopte, en de container ging mee ten onder. De twee juiste manieren: `podman logs -f mijn-api` (leest de logs die `common` opving; `Ctrl+C` stopt alleen de weergave) of `podman exec -it mijn-api sh` (nieuw proces, raakt PID 1 niet).

Waarom. `attach` maakt niets nieuws aan: het verbindt je terminal met de bestaande pijpen van het hoofdproces, signalen inbegrepen. Precies hetzelfde krijg je wanneer je de container op de voorgrond start.

Nuance. Er bestaat een nooduitgang: `Ctrl+P Ctrl+Q` koppelt los zonder te stoppen (als de container met `-it` gestart is), en `podman attach --sig-proxy=false` houdt signalen tegen. Maar het echte antwoord is simpel: gebruik `attach` niet om logs te lezen.

Voorbeeld.

```
podman logs -f --tail 20 mijn-api      # Ctrl+C: de container draait voort
podman attach --sig-proxy=false mijn-api # Ctrl+C wordt niet doorgegeven
```

Vraag 7 — 137, 143, 127

Antwoord. `api` (137): gedood door `SIGKILL` — ofwel een `stop` waarvan de respijtperiode verstreek, ofwel de OOM killer. Bevestigen: `podman inspect --format '{{.State.OOMKilled}}' api`, en daarna `podman events --since 1h | grep api` om na te gaan of er een `stop` was. `worker` (143): ontving `SIGTERM` en sloot af — een bewuste stop (uitrol, `podman stop`); bevestigen met `podman events` of `journalctl`. `batch` (127): het commando werd niet gevonden — de applicatie is nooit gestart (fout in de image of in `CMD`). Bevestigen: `podman logs batch` (melding `executable file not found`) en `podman inspect --format '{{json .Config.Cmd}}' batch`.

Waarom. Boven 128 is de code `128 + signaalnummer`. Daaronder is het de code die het programma zelf koos — of die van de shell/runtime wanneer het programma niet eens kon starten.

Nuance. Een `137` met `OOMKilled: false` en zonder `stop` in de events kan ook van een handmatige `kill -9` of van een orchestrator komen. En dat `worker` op hetzelfde moment 143

gaf als `api` 137, wijst op een gegroepeerde stop waarbij `api` niet netjes kon afsluiten: het symptoom van een shell vóór de applicatie (lab 04).

Voorbeeld.

```
podman inspect --format '{{.State.OOMKilled}} einde={{.State.FinishedAt}}' api
podman events --since 1h --filter container=api
```

Vraag 8 — Logs in een bestand

Antwoord. `podman logs` geeft alleen terug wat `conmon` opving op `stdout` / `stderr` van PID 1. Een applicatie die naar een bestand schrijft, passeert dat kanaal niet: er valt niets op te vangen. De map op de host mounten maakt het bestand wel leesbaar, maar blijft een slecht antwoord: de logs vallen buiten de tooling (`podman logs`, `journald`, verzamelagents), elke container verzint zijn eigen pad, niemand roteert de bestanden, en een verwijderde container laat weesbestanden achter.

Waarom. Het containermodel behandelt logs als een **stroom**: de engine vangt ze op, de tooling stuurt ze door (bestand, journal, Loki, Elastic). Een bestand in de container is lokale toestand, en dat botst met het wegwerpkarakter van een container.

Nuance. Spring Boot logt standaard naar de console: het volstaat om `logging.file.name` **niet** in te stellen. Wordt een bestandsformaat toch opgelegd, dan is de oplossing een *sidecar* of een agent die de stroom leest — geen mount.

Voorbeeld.

```
podman run -d --name l alpine sh -c 'echo zichtbaar; echo onzichtbaar > /tmp/app.log; sleep 100'
podman logs l                                # zichtbaar
podman exec l cat /tmp/app.log                 # onzichtbaar – alleen door binnen te gaan
```

Vraag 9 — `stop` / `start` tegenover `rm` / `run`

Antwoord. Na `stop` gevolgd door `start`: de gegevens zijn er **nog** — de schrijflaag van de container bestaat nog steeds, PostgreSQL vindt zijn bestanden terug. Na `rm` gevolgd door een nieuwe `run`: de gegevens zijn **weg** — `rm` vernietigde de schrijflaag, en de nieuwe container vertrekt opnieuw van de image.

Waarom. `stop` raakt alleen het proces; de container zelf (configuratie + laag) blijft bestaan. `rm` verwijdert het containerobject, laag inbegrepen.

Nuance. De image `postgres` declareert een `VOLUME`: de gegevens komen in een anoniem volume terecht dat de `rm` overleeft, maar nergens meer aan vasthangt — in de praktijk onbruikbaar. Het benoemde volume (lab 06) is de enige echte persistentie.

Voorbeeld.

```
podman run -d --name db -e POSTGRES_PASSWORD=x postgres:16-alpine
podman exec db psql -U postgres -c 'create table t(x int)'
podman stop db && podman start db && podman exec db psql -U postgres -c '\dt'    # t is er
podman rm -f -t 0 db && podman run -d --name db -e POSTGRES_PASSWORD=x postgres:16-alpine
podman exec db psql -U postgres -c '\dt'                                         # niets meer
```

Vraag 10 — `--restart=always` en de reboot

Antwoord. Onder Docker leest de daemon het beleid bij het opstarten opnieuw in en start hij de containers weer op. Onder Podman is er geen daemon: `common` past `--restart=always` toe zolang de container *in een levende sessie* bestaat, maar na een reboot draait er niets meer dat het beleid kan uitvoeren. De Podman-manier: een **Quadlet**-bestand (`/etc/containers/systemd/api.container`, of `~/.config/containers/systemd/` in rootless-modus) dat de container beschrijft, plus `systemctl enable --now api` — `systemd` start hem bij het booten en herstart hem bij falen.

Waarom. Podman wilde geen dienstbeheerder heruitvinden: Linux heeft er al een, `systemd`, met zijn afhankelijkheden, zijn logs en zijn opstart bij het booten. Een *restart policy* van Podman dekt alleen de duur van één sessie.

Nuance. In rootless-modus heb je daarbovenop `loginctl enable-linger <gebruiker>` nodig, zodat de diensten van die gebruiker ook zonder open sessie starten. Op een ontwikkelmachine onder WSL is dat zelden nodig: ontwikkelcontainers hoeven geen reboot te overleven.

Voorbeeld.

```
# ~/.config/containers/systemd/api.container
[Container]
Image=registry.intern/mijnapp/api:1.4.2
PublishPort=8080:8080
[Install]
WantedBy=default.target
```

```
systemctl --user daemon-reload && systemctl --user start api && systemctl --user status api
```

Vraag 11 — De logs van de eerste poging

Antwoord. Gewoon in `podman logs <container>`: bij elke herstart komen de logs **bovenop** die van de vorige uitvoering, op dezelfde container — de eerste poging staat dus bovenaan. `podman restart` wist ze evenmin, maar je verliest wel `.State.ExitCode` en `.State.FinishedAt` van de laatste uitvoering, en vooral: de container begint opnieuw aan zijn lus. Eerst kijken dus.

Waarom. Een automatische herstart start **dezelfde** container opnieuw (zelfde ID, zelfde schrijflaag, zelfde logbestand); er komt geen nieuwe. `podman events` levert daarbovenop de exacte tijdlijn (`died`, `restart`).

Nuance. Een `podman rm` (of `--rm`) verwijdert alles, logs inbegrepen. En een container die in een lus herstart, kan flink wat logs produceren: `--tail` en `--since` zijn je vrienden.

Voorbeeld.

```
podman logs --timestamps onstabiel | head -20          # de eerste uitvoering
podman events --since 10m --filter container=onstabiel
```

Vraag 12 — Van minst naar meest ingrijpend

Antwoord. (1) `podman inspect`: leest metadata, geen enkel effect — configuratie, toestand, OOM, host-PID. (2) `podman logs`: leest wat `common` al opving — wat de applicatie over zichzelf

vertelt. (3) `podman stats`: leest de cgroups — reëel CPU-, geheugen- en I/O-verbruik, zonder de container aan te raken. (4) `podman top`: voert aan hostzijde een `ps` uit op de PID's van de container — welk proces verbruikt, welke threads. (5) `podman exec`: start een proces **in** de container — het meest ingrijpend, maar de enige weg naar een `jstack` of een `curl localhost:8080/actuator`.

Waarom. De eerste vier kijken van buitenaf toe, via de engine of de kernel; alleen `exec` verandert iets binnenin (een proces extra, resources verbruikt in de cgroup van de container).

Nuance. Met de host-PID uit `inspect` kun je verder gaan zonder `exec`: `cat /proc/<pid>/status`, `strace -p <pid>` — in rootless-modus is de container immers een proces van jouw gebruiker. En een *distroless* image heeft geen shell: daar is `exec` niet eens mogelijk (lab 05).

Voorbeeld.

```
podman stats --no-stream api
podman top api pid,pcpu,comm
podman exec api jcmd 1 Thread.print | head -50
```

Vraag 13 — API en database in dezelfde container

Antwoord. Drie gevolgen. (1) **Eén enkele PID 1**: je hebt een toezichthouder (`supervisord`) nodig om twee processen overeind te houden, en sterft de database, dan merkt de container het niet — of omgekeerd: de API sterft en sleurt de database mee. (2) **Gekoppelde levenscyclus**: de API opnieuw uitrollen betekent ook PostgreSQL herstarten, met al zijn verbindingen en zijn cache. (3) **Resources en observeerbaarheid op één hoop**: één geheugenlimiet, één door elkaar gemengde logstroom, en de API schalen kan niet zonder ook de database te dupliceren.

Waarom. Een container is ontworpen rond *één* hoofdproces waarvan het leven samenvalt met dat van de container. Twee processen betekent twee levenscycli in een object dat er maar één heeft.

Nuance. Podman heeft een object voor "meerdere containers die samen moeten leven": de **pod** (`podman pod create`). Die deelt netwerk en levenscyclus, maar houdt één container per proces — hetzelfde concept als bij Kubernetes. Dat is het correcte antwoord op de wens "eenvoudig kunnen starten".

Voorbeeld.

```
podman pod create --name stack -p 8080:8080
podman run -d --pod stack --name db -e POSTGRES_PASSWORD=x postgres:16-alpine
podman run -d --pod stack --name api mijn-api:1.0 # bereikt db op localhost:5432
```

Vraag 14 — `--rm` en productie

Antwoord. Voor een eenmalig commando voorkomt `--rm` dat dode containers zich opstapelen. Voor een dienst in productie vernietigt het bij het afsluiten precies wat je na een incident nodig hebt: de **logs**, de **exitcode**, de **schrijflaag** (tijdelijke bestanden, *heap dump*) en de mogelijkheid om nog `podman inspect` te doen. De container is dood en er valt niets meer te onderzoeken. De combinatie met `--restart` is in wezen tegenstrijdig: `--rm` verwijdert de container bij het afsluiten, `--restart` wil hem op datzelfde moment herstarten — wat je net gewist hebt, kun je niet herstarten. Podman weigert de combinatie dan ook expliciet.

Waarom. De `Exited`-container is je bewijsmateriaal. Een dienst die om 3 uur 's nachts crashte, moet je om 9 uur nog kunnen onderzoeken.

Nuance. Orchestrators (Kubernetes, Compose) ruimen beëindigde containers zelf op, met een vertraging en met logretentie. `--rm` blijft perfect voor tool-containers: compilatie, databasemigratie, een interactieve `psql`.

Voorbeeld.

```
podman run --rm -d --restart=always nginx:alpine
# Error: the --rm option conflicts with --restart, when the restartPolicy is not "" and "no"
```